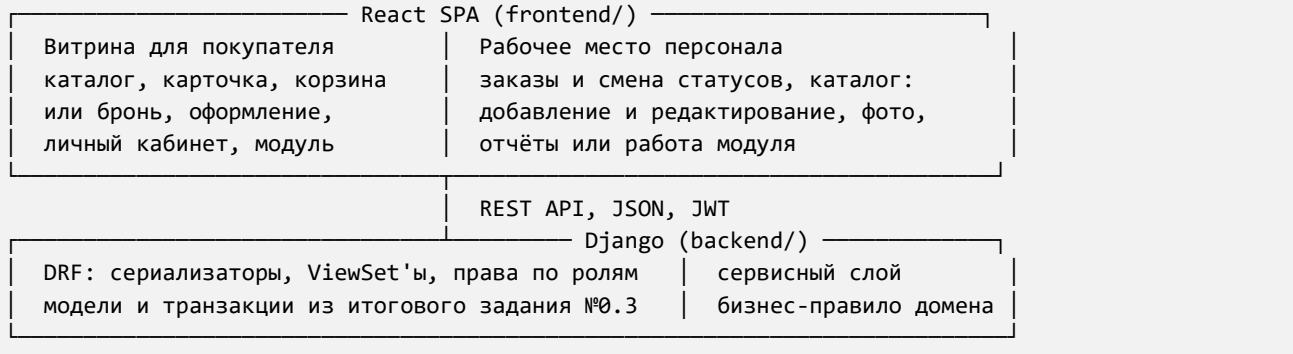
Итоговое задание блока React: платформа на React + Django

Срок: 6 недель от выдачи задания, 4 контрольные точки и защита (даты объявляет преподаватель)

Что проверяется: всё, что пройдено в треке 0: Django и сервисный слой (лабораторные №0–0.3), React (лабораторные №0.4–0.7), соединение React с Django через REST API и JWT (лабораторная №0.8). Плюс самостоятельное освоение нескольких «взрослых» инструментов фронтенда на выбор.

1. Что нужно сделать

Каждый студент превращает свой проект из **итогового задания лабораторной №0.3** (магазин или сервис по шифру Д<домен>-М<модуль>) в **платформу** из двух частей:



Главное отличие от итогового задания №0.3: **персонал работает не в admin panel, а в React-приложении**, а все правила (остатки, бизнес-правило домена, правило модуля, права) по-прежнему проверяет **сервер**. Интерфейс React только помогает пользователю не ошибиться, но не является защитой (лабораторная №0.8, часть 1.6).

Шаблоны Django из итогового задания №0.3 можно оставить (они не мешают), удалить или не развивать: оценивается API и React-приложение.

1.1 Вариант

Шифр остаётся тем же, что в итоговом задании лабораторной №0.3: домен — по первой букве фамилии, модуль — по первой букве имени, с теми же правилами разрешения совпадений (таблицы — в лабораторной №0.3, п. 11.2, домены — п. 11.4, модули — п. 11.5).

- Если итоговое задание №0.3 было выполнено и защищено — его проект и становится бэкендом.
- Если не было выполнено — шифр определяется по тем же таблицам, бэкенд делается заново. За основу можно взять код лабораторных №0.2, №0.3 и №0.8, но не тему «магазин компьютерной техники».
- Своя тема — по тем же условиям, что в п. 11.2 лабораторной №0.3, согласуется до КТ1. Если своя тема уже была согласована в №0.3, она сохраняется.

2. Обязательная часть

2.1 Бэкенд (Django + DRF)

№	Требование
Б1	Репозиторий из двух папок: `backend/` (Django) и `frontend/` (React), общий `README.md`. Отдельные `requirements.txt` и `package.json`, `.gitignore` для обоих (нет `venv/`, `node_modules/`, `dist/`, `db.sqlite3`, `media/`, `.env*` кроме `.env.example`)
Б2	REST API на Django REST Framework: сериализаторы, ViewSet'ы или generic-представления, адреса под `/api/`. В README — таблица всех адресов API: метод, адрес, кто имеет доступ, что делает
Б3	Вход по JWT (SimpleJWT): получение и обновление токена, регистрация, `GET /api/auth/me/` (с ролью пользователя: покупатель или персонал). По умолчанию все адреса закрыты (`IsAuthenticated`), открытые разрешены явно
Б4	**Права по ролям:** собственный класс прав для персонала (например, `IsManager`, см. п. 6), адреса рабочего места персонала покупателю отвечают 403. Объектный уровень: покупатель видит и меняет только своё (чужое — 404)
Б5	Каталог в API: постраничный вывод на сервере, фильтр по категории и по атрибуту своего домена, поиск без учёта регистра (в том числе кириллица), сортировка — не меньше двух вариантов. Можно `django-filter` или собственная фильтрация в `get_queryset`
Б6	Оформление заказа (брони, записи) через **сервисный слой** в одной транзакции: проверка доступности, списание остатков, снимок цены, откат при ошибке. Клиент присылает только то, что выбирает пользователь; цены, суммы и статусы считает сервер
Б7	API рабочего места персонала: список заказов с фильтром по статусу, смена статуса **только допустимыми переходами** (недопустимый — 400 с понятным сообщением), создание и редактирование объектов каталога **с загрузкой фото** (multipart)
Б8	Бизнес-правило домена и правило модуля проверяются на сервере: нарушение через прямой запрос к API (минуя React) получает 400/403 с понятным сообщением, а не 500 и не «тихий» успех
Б9	CORS для адреса фронтенда при разработке (`django-cors-headers`), ошибки валидации — в формате DRF (`{"поле": ["…"]}` или `{"detail": "…"}`)
Б10	Команда `python manage.py seed_demo` — не меньше 20 объектов каталога, справочники, демонстрационные пользователи: покупатель и сотрудник (логины и пароли — в README). Повторный запуск не создаёт дублей
Б11	Не меньше **15 тестов API** (`APITestCase`), все проходят. Обязательно: права (аноним, покупатель, персонал, чужой объект), оформление заказа и откат, бизнес-правило домена, правило модуля, недопустимый переход статуса, подмена цены клиентом игнорируется

2.2 Фронтенд (React)

№	Требование
Ф1	Vite + React 19, React Router 8. Не меньше **10 маршрутов**, среди них: с параметрами (`/products/:id`), вложенные каркасы (отдельные каркасы для витрины и для рабочего места персонала), страница 404
Ф2	**Защищённые маршруты по ролям:** страницы покупателя требуют входа, рабочее место персонала — роли персонала (покупатель получает страницу «Нет доступа»). После входа — возврат на страницу, куда шёл пользователь. Проверка входа при загрузке не «выбрасывает» вошедшего пользователя (лабораторная №0.8, часть 9)
Ф3	Состояние приложения: контексты (вход, корзина или бронь) с собственными хуками; не меньше **двух собственных хуков**, кроме хуков контекстов (например, `useDebounce`, `useFetch`, `useDocumentTitle`)
Ф4	Каталог: условия отбора, поиск, сортировка и номер страницы — в адресе (`useSearchParams`); данные — с сервера постранично (Б5); поиск с задержкой (debounce) и защитой от гонки ответов; состояния «загрузка» (заготовки карточек, а не просто текст), «ошибка» с повтором, «ничего не найдено»
Ф5	Формы: управляемые поля, проверка в браузере, повторяющая серверную; **ошибки сервера показываются под соответствующими полями**; на время отправки кнопка блокируется, двойная отправка невозможна
Ф6	Корзина или бронь сохраняется в `localStorage`; оформление с подсказками по бизнес-правилу домена **до** отправки (например, предупреждение о рецептурном препарате ещё в корзине для Д6); после успеха — страница заказа, при ошибке сервера данные формы не теряются
Ф7	Личный кабинет: история заказов, страница заказа, действия по правилам (отмена и т.п.)
Ф8	Рабочее место персонала в React: заказы с фильтром по статусу и сменой статуса (доступны только допустимые переходы, но сервер всё равно проверяет), список объектов каталога, форма добавления и редактирования **с загрузкой фото и предпросмотром**
Ф9	Работа с API в одном модуле: JWT в заголовке, автоматическое обновление `access` по `refresh` **одним запросом**, даже если несколько запросов получили 401 одновременно; при истёкшем `refresh` — аккуратный выход и переход на страницу входа
Ф10	Граница ошибок (error boundary) на уровне каркаса: ошибка в одной странице показывает сообщение «Что-то пошло не так» и ссылку на главную, а не белый экран
Ф11	Прокси Vite для `/api` при разработке, адрес API через `VITE_API_URL` с примером в `frontend/.env.example`; `npm run lint` без ошибок и предупреждений, `npm run build` проходит
Ф12	Интерфейс работает на ширине телефона (360 px) без горизонтальной прокрутки; у полей форм есть подписи (`<label>`); заголовок вкладки меняется по страницам
Ф13	**Модуль по варианту** — в API и в интерфейсе (п. 3)

3. Модуль в платформе

Модуль тот же, что в итоговом задании №0.3, но теперь у него есть интерфейс на React и API. Серверные требования из п. 11.5 лабораторной №0.3 сохраняются (кроме тех, что относятся к admin panel и шаблонам), плюс:

M1. Отзывы и рейтинг.

- На странице товара — список одобренных отзывов с постраничной подгрузкой («Показать ещё») и форма отзыва, которая видна только тем, кому сервер разрешает оставить отзыв (поле `can_review` в ответе API товара).
- Звёздный рейтинг в форме — свой компонент, работающий с клавиатуры (стрелки, Tab).
- В рабочем месте персонала — очередь отзывов на модерацию с кнопками «Одобрить» и «Отклонить»; после действия отзыв исчезает из списка сразу, без перезагрузки страницы.

M2. Промокоды и скидки.

- Поле промокода в корзине: проверка через API (`POST /api/promo/check/` с кодом и суммой корзины) с выводом скидки или причины отказа; применённый промокод сохраняется вместе с корзиной.
- Пересчёт при изменении корзины: если сумма стала меньше минимальной, промокод помечается как неприменимый с объяснением.
- В рабочем месте персонала — список промокодов с числом использований, создание и деактивация.

M3. Избранное и уведомления о поступлении.

- Кнопка «В избранное» на карточке и странице товара с **оптимистичным обновлением**: отметка меняется сразу, а при ошибке сервера возвращается обратно с сообщением.
- Счётчик непрочитанных уведомлений в шапке обновляется сам каждые 30 секунд (`setInterval` в эффекте с очисткой) и при возврате на вкладку (событие `visibilitychange`); страница уведомлений, «прочитать всё».
- В рабочем месте персонала пополнение остатка товара с 0 создаёт уведомления — это проверяется тестом API.

M4. Отчёты для менеджера.

- Страница отчётов в рабочем месте персонала: выбор периода (по умолчанию 30 дней), период хранится в адресе.
- График выручки по дням (библиотека Recharts или аналог) и диаграмма заказов по статусам; таблица топ-10 товаров с сортировкой по столбцам на клиенте.
- Скачивание CSV за период: файл формирует сервер, React скачивает его с токеном (через `fetch` и `Blob`, а не простой ссылкой).

4. Расширения на выбор

Каждый выбирает и выполняет **не меньше трёх** расширений (обязательная часть оценки, п. 8). Каждое следующее после трёх — бонус до +10 баллов. Выбор фиксируется в README на КТ2.

№	Расширение	Что должно быть
P1	**TypeScript**	весь фронтенд на TypeScript (`.tsx`, шаблон `react-ts`), режим `strict`, типы для всех ответов API, `npm run build` без ошибок типов, без `any` (кроме обоснованных случаев с комментарием)
P2	**TanStack Query**	все запросы к API — через `useQuery`/`useMutation`: кэш, повторное использование данных между страницами, инвалидация после изменений (например, после смены статуса заказа список обновляется сам), оптимистичное обновление хотя бы в одном месте
P3	**Режим данных React Router**	`createBrowserRouter` и `RouterProvider`; загрузка данных страниц в `loader`, отправка форм в `action`, ошибки — в `errorElement`, индикатор перехода через `useNavigation`
P4	**Тесты фронтенда**	Vitest + React Testing Library, не меньше 12 тестов: компоненты, формы с проверкой, собственные хуки, защищённый маршрут; ответы API подменяются (MSW или моки `fetch`); `npm test` в README
P5	**E2E-тест**	Playwright: сценарий покупателя (регистрация → поиск → корзина → заказ → личный кабинет) и сценарий сотрудника (смена статуса заказа) проходят на запущенных серверах; запуск — одной командой из README
P6	**Разбиение кода**	рабочее место персонала и тяжёлые страницы загружаются лениво (`React.lazy`, `Suspense`), в выводе `npm run build` видно отдельные файлы, в README — размеры до и после
P7	**Документация API**	`drf-spectacular`: схема OpenAPI и страница Swagger UI по адресу `/api/docs/`, у всех адресов описаны параметры и ответы, примеры запросов; описания на русском
P8	**Корзина на сервере**	у вошедшего пользователя корзина хранится в базе и доступна с любого устройства; при входе корзина из `localStorage` объединяется с серверной (правило объединения описано в README и покрыто тестом)
P9	**Обновления в реальном времени**	статус заказа у покупателя меняется на экране без перезагрузки, когда сотрудник его поменял: Django Channels + WebSocket или Server-Sent Events. Опрос сервера по таймеру не засчитывается (для М3 он входит в модуль)
P10	**Тёмная тема**	цвета через CSS-переменные, тема по

		настройке системы (`prefers-color-scheme`) и переключатель, выбор запоминается; контраст текста не ниже 4,5:1 в обеих темах
P11	**Доступность**	Lighthouse → Accessibility не ниже 95 на главной, карточке и оформлении; всё приложение проходится с клавиатуры (видимый фокус, модальные окна возвращают фокус); отчёт Lighthouse в репозитории
P12	**Бесконечная прокрутка**	каталог подгружается при прокрутке (`IntersectionObserver`) вместо номеров страниц, без дублей и пропусков при быстрой прокрутке; позиция прокрутки восстанавливается после возврата со страницы товара
P13	**Docker Compose**	`docker compose up` поднимает PostgreSQL, Django (gunicorn) и nginx, который раздаёт сборку React и проксирует `/api` и `/media`; переменные окружения в `.env.example`; связь с блоком Docker основного курса
P14	**CI**	GitHub Actions (или аналог в GitVerse/GitFlic): на каждый push — тесты Django, `npm run lint`, `npm run build` (и `npm test`, если выбран P4); зелёная отметка у последнего коммита

Расширение засчитывается, только если студент на защите объясняет, как оно устроено, и может внести небольшое изменение (п. 7).

5. Контрольные точки

Точка	Срок	Что должно быть в репозитории
КТ1. Контракт API	неделя 1	структура `backend/` + `frontend/`; бэкенд из №0.3 переведён в `backend/`; README: роли, 8–12 сценариев вида «Покупатель может…», «Сотрудник может…», **таблица адресов API** (Б2) с примерами JSON для 3–4 главных адресов; каркас React с маршрутами-заглушками; преподаватель приглашён в репозиторий
КТ2. Каталог и вход	неделя 3	API каталога (Б5), JWT и регистрация (Б3); витрина React: каталог с фильтрами в адресе, карточка, вход, регистрация (Ф1–Ф5 для этих страниц); выбор расширений записан в README; не меньше 6 тестов API
КТ3. Заказы и персонал	неделя 5	оформление (Б6, Ф6), личный кабинет (Ф7), рабочее место

		персонала (Б7, Ф8), права (Б4, Ф2), бизнес-правило домена (Б8); не меньше 12 тестов API
КТ4. Модуль и расширения	неделя 6, начало	модуль (Ф13), не меньше трёх расширений, граница ошибок, адаптивность; не меньше 15 тестов API
Защита	неделя 6	итоговая версия, тег `v2.0` в Git

На каждой точке преподаватель смотрит репозиторий и даёт замечания. Точка пройдена, если к сроку в репозитории есть перечисленное.

6. Подсказки

Это не готовые решения, а отправные точки. Документация: https://www.django-rest-framework.org, https://reactrouter.com, https://react.dev.

Права для персонала (`backend/accounts/permissions.py`):

```
from rest_framework.permissions import BasePermission


class IsManager(BasePermission):
    """Доступ только для группы «Менеджеры» и администраторов."""

    def has_permission(self, request, view):
        user = request.user
        return user.is_authenticated and (
            user.is_superuser or user.groups.filter(name="Менеджеры").exists()
        )
```

Подключается в представлениях рабочего места: `permission_classes = [IsManager]`. Чтобы React знал роль, добавьте в сериализатор `GET /api/auth/me/` поле вроде `is_manager = serializers.SerializerMethodField()`.

Задержка для поиска (`frontend/src/hooks/useDebounce.js`):

```
import { useEffect, useState } from 'react'

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(timer)
  }, [value, delay])

  return debounced
}
```

Поле поиска управляет строкой `query`, а запрос к API зависит от `useDebounce(query)`: он уйдёт, только когда пользователь перестанет печатать на 300 мс.

Загрузка фото. Файлы отправляют не JSON, а `FormData`. Заголовок `Content-Type` в этом случае **не указывают**: браузер сам поставит `multipart/form-data` с границей между частями. DRF по умолчанию принимает такие запросы.

```
export async function uploadProductImage(productId, file, token) {
  const formData = new FormData()
  formData.append('image', file)
  const response = await fetch(`/api/manager/products/${productId}/`, {
    method: 'PATCH',
    headers: { Authorization: `Bearer ${token}` },
    body: formData,
  })
  if (!response.ok) {
    throw new Error(`Ошибка загрузки: ${response.status}`)
  }
  return response.json()
}
```

Предпросмотр выбранного файла до загрузки: `URL.createObjectURL(file)` (и `URL.revokeObjectURL` в очистке эффекта).

Одно обновление токена на всех (Ф9). Храните Promise текущего обновления в переменной модуля `api.js`: первый запрос, получивший 401, создаёт его, остальные ждут тот же Promise.

Граница ошибок (Ф10). В React граница ошибок — это классовый компонент с методом `static getDerivedStateFromError`, либо готовый пакет `react-error-boundary`. Она перехватывает ошибки отрисовки дочерних компонентов, но не ошибки в обработчиках событий и асинхронном коде — их обрабатывайте сами, как в лабораторной №0.6.

Допустимые переходы статусов (Б7). Опишите их один раз, например словарём `{"new": ["confirmed", "cancelled"], ...}` в модели или сервисе, и отдавайте в API заказа поле `allowed_transitions`: React покажет только доступные кнопки, а сервер проверит по тому же словарю.

7. Как проходит приёмка

Защита занимает около 20 минут и идёт по одному протоколу для всех.

1. **Развёртывание с нуля по README** (не больше 15 минут). Клонирование в пустую папку и только команды из README:

   - бэкенд: окружение, `pip install -r requirements.txt`, `migrate`, `seed_demo`, `runserver`;
   - фронтенд: `npm ci`, `npm run dev`.

   Если проект не запускается по инструкции, защита переносится.

2. **Автоматические проверки:** `python manage.py check`, `python manage.py makemigrations --check`, `python manage.py test`, `npm run lint`, `npm run build` (и `npm test`, если выбран Р4). Если что-то не проходит, защита переносится.

3. **Сценарий покупателя в React:** регистрация, поиск с фильтрами (ссылку на результаты открыть в новой вкладке), корзина, оформление, личный кабинет, действие с заказом, модуль.

4. **Сценарий сотрудника в React:** вход демонстрационным сотрудником, обработка заказа (только допустимые переходы), добавление объекта каталога с фото — новый объект сразу виден на витрине.

5. **Негативные проверки — через интерфейс и напрямую через API** (DevTools или `curl`):

   - чужой заказ по прямой ссылке и по адресу API — 404;
   - адреса рабочего места с токеном покупателя — 403, в React — «Нет доступа»;
   - заказ с количеством больше остатка, нарушение бизнес-правила домена, недопустимое применение модуля, недопустимый переход статуса — отправленные **напрямую в API, минуя React**: понятный 400/403, данные не изменились;
   - подмена цены или суммы в запросе — игнорируется;
   - истёкший `access` (преподаватель может попросить уменьшить срок жизни токена) — приложение продолжает работать;
   - Django остановлен — понятное сообщение, а не белый экран.

6. **Вопросы по коду.** Преподаватель открывает 2–3 места в коде фронтенда и бэкенда (обязательно — одно из расширений) и просит объяснить, что там происходит и почему.

7. **Доработка на месте** (до 10 минут): например, добавить поле в модель, вывести его в API и на странице React; добавить вариант сортировки, который проходит через адрес, API и базу; изменить правило перехода статуса и поправить тест.

Правила:

- код лабораторных и итогового задания №0.3 можно и нужно использовать как основу;
- документацией, примерами и ИИ-ассистентами пользоваться можно, но студент отвечает за каждую строку. Если студент не может объяснить фрагмент своего кода, баллы за соответствующий критерий не засчитываются;
- история Git отражает работу: не меньше 25 коммитов, распределённых по контрольным точкам, с понятными сообщениями. Один большой коммит в конце — 0 баллов за критерий «Git и качество кода» и дополнительные вопросы на защите;
- совпадающий код в работах разных студентов (кроме кода лабораторных) — повторная защита обеих работ с расширенной доработкой на месте.

8. Оценивание

Критерий	Баллы
Развёртывание по README, `seed_demo`, демонстрационные пользователи, `.env.example`	5
API: контракт в README, сериализаторы, постраничный вывод, фильтры, поиск, ошибки в формате DRF (Б2, Б5, Б9)	10
Вход и права: JWT, роли, 401/403/404, обновление токена одним запросом, защищённые маршруты (Б3, Б4, Ф2, Ф9)	10
Заказы: транзакция, остатки, снимок цены, бизнес-	10

правило домена на сервере и в интерфейсе (Б6, Б8, Ф6, Ф7)	
Рабочее место персонала в React: переходы статусов, каталог с фото (Б7, Ф8)	10
Фронтенд: маршруты и каркасы, контексты и свои хуки, состояние в адресе, загрузка/ошибки/гонки, формы с серверными ошибками, граница ошибок, адаптивность (Ф1, Ф3–Ф5, Ф10–Ф12)	15
Модуль по варианту в API и в React (Ф13)	10
Расширения: три на выбор (по 5)	15
Тесты API: не меньше 15, осмысленные, все проходят (Б11)	5
Git и качество кода: история коммитов, понятные имена, нет мёртвого кода, линтер без замечаний	5
Защита: ответы на вопросы и доработка на месте	5
Итого	**100**
Бонус: каждое расширение сверх трёх	до +10

Баллы	**Оценка**
85 и больше	отлично
70–84	хорошо
55–69	удовлетворительно
меньше 55	работа возвращается на доработку

Пропуск контрольной точки — минус 5 баллов за каждую (не больше 15 в сумме). Работа не принимается к защите, если не выполнены пункты 1 и 2 протокола приёмки.

9. Чек-лист перед защитой

- ☐ Клонировал репозиторий в пустую папку и поднял оба проекта строго по README
- ☐ `check`, `makemigrations --check`, `test`, `npm run lint`, `npm run build` — без ошибок
- ☐ Демонстрационные покупатель и сотрудник входят, логины и пароли в README верные
- ☐ Ссылка на каталог с фильтрами открывается в новой вкладке с теми же результатами
- ☐ Обновление любой страницы (`F5`) не ломает её и не выбрасывает вошедшего пользователя
- ☐ Покупатель не попадает в рабочее место ни через интерфейс, ни через API
- ☐ Бизнес-правило домена и правило модуля нарушены прямым запросом к API — сервер отказал
- ☐ При остановленном Django и при ошибке в компоненте — понятное сообщение, а не белый экран
- ☐ Страницы читаются на ширине 360 px
- ☐ Могу объяснить каждое выбранное расширение и показать, где оно в коде
- ☐ В README: описание, роли, сценарии, адреса API, команды запуска, выбранные расширения, демо-пользователи
- ☐ Поставлен тег `v2.0`